

Subhakamana Store Management System

Progress PDF Report - Backend, PostgreSQL Readiness, CI Validation, and ERP API Bridge

Area	Current Status
Customer Website / ERP Prototype	Functional front-end prototype with local demo persistence and improving backend API bridge.
Module 1 Database	Prisma/PostgreSQL schema, migration, hardened seed, Docker Compose plan, and verifier prepared.
Module 2 Backend API	Authentication, sessions, RBAC, Product API, Inventory API, and tests prepared.
Live PostgreSQL	Blocked locally because no Docker/PostgreSQL runtime is available in this sandbox. CI workflow prepared.
Latest Test Result	Module 1 verifier passed. Module 2: 31 passed, 1 skipped. Skipped test requires TEST_DATABASE_URL.

Prepared for: Subhakamana Store project demonstration and production-conversion tracking.

Report date: 17 July 2026

Project type: Front-end ERP/e-commerce prototype being migrated toward backend-backed production architecture.

1. Executive Summary

The Subhakamana Store Management System has moved beyond a static front-end prototype into a structured production-conversion phase. The current project includes a customer-facing shopping website, an Admin ERP interface, a separate physical POS concept, documentation, workflow reports, UML assets, local launch files, and a growing backend foundation.

The most recent work focused on PostgreSQL readiness, backend API foundations, CI validation, and connecting the ERP product and inventory workflows to backend endpoints. Product List, Add Product, Inventory View, and Receive Stock now have backend API bridge logic in the ERP prototype, with local fallback behavior for demo use.

A true live PostgreSQL validation has not yet been completed locally because the current Codex sandbox cannot start PostgreSQL: Docker/PostgreSQL tools are unavailable, and a temporary source-built PostgreSQL server was blocked by macOS shared-memory restrictions. To solve this, a GitHub Actions workflow has been added to run the PostgreSQL validation in CI using temporary non-production databases and encrypted secrets.

2. Current System Position

Component	Progress	Notes
Customer Website	Prototype complete / improving	Fashion-store UI, products, cart, checkout concepts, trust sections, QR payment placement, and responsive structure.
Admin ERP	Advanced prototype	Inventory, products, orders, delivery, payments, reports, returns, suppliers, and manual/help sections.
Cashier POS	Prototype separated	Fast physical-sale flow exists conceptually and in UI; production checkout backend is still future work.
Database Foundation	Prepared	Prisma schema, migration SQL, seed, env examples, Docker Compose plan, and verification scripts exist.
Backend API	Foundation complete	Auth, sessions, RBAC, Product API, Inventory API, repository pattern, and route tests exist.
PostgreSQL Live Validation	Blocked locally	Needs Docker/Postgres.app or CI secrets to run real migrations, seed, rollback, and concurrency tests.

3. Completed Backend and Database Work

Module 1 - Database Foundation

- Created a PostgreSQL/Prisma database foundation for production conversion.
- Prepared Prisma schema and initial migration SQL.
- Added hardened seed process for roles, permissions, role-permission mappings, first administrator, categories, and store settings.
- Added environment examples for DATABASE_URL, TEST_DATABASE_URL, seed admin values, and runtime settings.
- Added module verification script. Latest result: Database module verification passed.

Module 2 - Backend API Foundation

- Built API foundation with health endpoints, consistent success/error response shape, authentication logic, sessions, and server-side RBAC.
- Added Prisma user/session repositories and Product/Inventory repositories.
- Added Product API and Inventory API routes for listing products, creating products, listing inventory, and receiving stock.
- Kept in-memory fallback mode for local demo use when DATABASE_URL is not configured.

- Fixed backend startup so node src/server.js starts the API correctly.
- Fixed refresh cookie path to /api/v1 so protected Product and Inventory routes can receive session cookies.
- Updated CORS handling for credentialed browser requests.

4. ERP API Bridge Progress

The ERP front-end now contains backend bridge functions for the key localStorage-to-API migration workflows. These bridge functions call the backend first and fall back to local demo data when the backend is unavailable, keeping demonstrations usable while production conversion continues.

Workflow	Backend Endpoint	Current Behavior
Product List	GET /api/v1/products	Loads backend products and merges with local inventory view when available.
Inventory View	GET /api/v1/inventory	Loads backend inventory rows and updates local product stock state.
Add Product	POST /api/v1/products	Attempts backend create first, then local fallback if API is unavailable.
Receive Stock	POST /api/v1/inventory/receive-stock	Attempts backend stock receipt first, records movement, then local fallback if unavailable.

5. Testing and Validation Status

Test Area	Latest Result	Meaning
Module 1 verifier	Passed	Database module files and expected structure verified.
Module 2 unit/route tests	31 passed, 1 skipped	Backend auth, sessions, RBAC, Product/Inventory routes, and repository behavior pass offline tests.
Backend smoke test	Passed in in-memory mode	Health, Product List, Inventory View, Add Product, and Receive Stock routes responded correctly.
PostgreSQL integration test	Skipped	Requires TEST_DATABASE_URL and a real non-production PostgreSQL server.
Live migrations and seed	Not executed locally	Blocked by unavailable PostgreSQL runtime in this sandbox.

6. PostgreSQL Environment Findings

Several methods were checked for running a real local PostgreSQL database. Docker, Docker Compose, Podman, Colima, Homebrew, and local PostgreSQL binaries were not available in the current shell. The official Postgres.app DMG downloaded but could not be mounted in this environment. PostgreSQL 16.10 was successfully compiled from official source, but database initialization failed because the sandbox blocks required shared-memory system calls such as shmget and shm_open.

Because of this, no local claim is made for migration safety, seed safety, transaction rollback safety, concurrency safety, or persistence after restart. Those claims require a running PostgreSQL instance and passing live tests.

7. GitHub Actions CI Workflow

A GitHub Actions workflow has been configured to perform PostgreSQL validation outside the local sandbox. The workflow starts a temporary PostgreSQL 16 service, creates separate temporary development and test databases,

applies migrations, runs the hardened seed, verifies seeded roles and permissions, generates Prisma Client, executes the gated PostgreSQL integration test, runs Module 1 and Module 2 checks, and drops the temporary databases afterward.

Required GitHub Secrets

- CI_POSTGRES_PASSWORD
- CI_SEED_ADMIN_PASSWORD
- CI_SESSION_SECRET

These values must be stored as encrypted GitHub Actions secrets. They must be CI-only and must never point to production credentials or a production database.

8. Remaining Limitations

- Real PostgreSQL migrations have not yet been applied in a live non-production database from this local environment.
- The hardened seed has not yet been verified against a live PostgreSQL database locally.
- Rollback and concurrent inventory-update behavior are not yet proven against PostgreSQL.
- Protected frontend API use will require a real login/session flow once the ERP is fully moved from local demo mode to backend-backed mode.
- POS checkout, website checkout, payments, delivery integration, and full production deployment remain future backend phases.

9. Recommended Next Steps

Priority	Step	Reason
1	Add GitHub encrypted secrets and push CI workflow.	Allows PostgreSQL validation to run using a real temporary CI database.
2	Run CI PostgreSQL workflow and inspect result.	Confirms migrations, seed, Product/Inventory APIs, rollback tests, and concurrency behavior.
3	Fix any failing live PostgreSQL test findings.	Only live DB testing can expose real transaction or constraint issues.
4	Add frontend login/session handling for protected backend APIs.	Needed for Product/Inventory API access when protectBusinessRoutes is enabled.
5	Continue production conversion module by module.	Next phases: POS checkout, online orders, payment verification, returns, reports, backups, and monitoring.

10. Conclusion

The project is in a strong prototype-to-production transition stage. The visual ERP and customer website are usable for demonstration, and the backend foundation is now structured enough to support real database-backed Product and Inventory workflows. The main blocker is not the application design; it is the availability of a real non-production PostgreSQL runtime for final validation.

Once GitHub CI secrets are configured or a dedicated development PostgreSQL database is provided, the next milestone is to run live migrations, seed validation, authentication tests, Product/Inventory tests, rollback checks, and concurrency checks. After that passes, the project can move confidently into deeper production modules such as POS checkout, online checkout, payments, delivery, reporting, backup, and monitoring.